

Kubernetes Hackathon

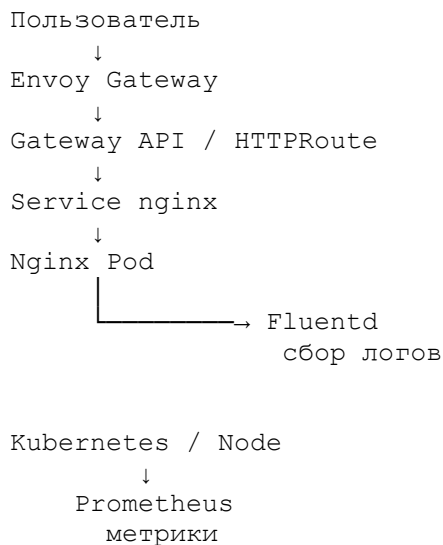
Страница 1. Архитектура и состав решения

В рамках проекта развернут одноузловой Kubernetes-кластер на Ubuntu Server 24.04.

Используемые технологии:

- Kubernetes — v1.36.5
- Способ развертывания Kubernetes — kubectl
- Операционная система — Ubuntu Server 24.04.4 LTS
- Container Runtime — containerd 2.2.1
- Сетевой плагин — Calico v3.33.0
- Реализация Gateway API — Envoy Gateway v1.9.2
- Веб-приложение — Nginx 1.27
- Мониторинг — Prometheus, установленный через Helm
- Дополнительные компоненты мониторинга — Node Exporter и kube-state-metrics
- Сбор логов — Fluentd 1.18
- Автоматизация — Shell-скрипты

Архитектура решения:



Пользовательский HTTP-запрос поступает в Envoy Gateway. Созданный HTTPRoute направляет запрос в Kubernetes Service `nginx`, который передает его в Nginx Pod.

Prometheus используется для мониторинга Kubernetes-инфраструктуры. Fluentd собирает контейнерные логи Nginx.

Тестирование решения проводилось в локальной виртуальной машине VirtualBox с Ubuntu Server 24.04.

1. Kubernetes

Что реализовано:

На Ubuntu Server 24.04 развернут одноузловой Kubernetes-кластер версии 1.36.5.

Кластер создан с помощью `kubeadm`. В качестве Container Runtime используется `containerd`, а для организации сети Kubernetes установлен `Calico`.

Почему выбран такой вариант:

`kubeadm` позволяет развернуть полноценный Kubernetes-кластер и соответствует рекомендуемому способу развертывания в задании.

Как проверить:

```
kubectl get nodes -o wide
```

Узел должен находиться в состоянии `Ready`.

2. Веб-приложение

Что реализовано:

В Kubernetes развернуто простое веб-приложение на базе `Nginx 1.27`.

Созданы Kubernetes `Deployment` и `Service`. `Service` предоставляет доступ к `Nginx` внутри кластера по порту 80.

Почему выбран Nginx:

Для задания достаточно простого приложения с детерминированным HTTP-ответом. `Nginx` позволяет легко проверить работоспособность всей цепочки маршрутизации.

Как проверить:

```
kubectl get pods,svc -n hackathon
```

`Nginx Pod` должен находиться в состоянии `Running`.

3. Gateway API

Что реализовано:

Для публикации приложения используется `Envoy Gateway`.

Созданы:

```
GatewayClass: eg
Gateway: hackathon-gateway
HTTPRoute: nginx-route
```

`HTTPRoute` направляет входящие HTTP-запросы в `Service nginx` на порт 80.

Как проверить:

```
kubectl get gatewayclass
kubectl get gateway,httproute -n hackathon
```

```
kubectl get svc -n envoy-gateway-system
```

В локальном окружении VirtualBox отсутствует облачный LoadBalancer, поэтому внешний IP Envoy остается в состоянии pending. Доступ к приложению выполняется через NodePort Envoy Service.

В тестовом окружении использовался NodePort 30252.

Проверка:

```
curl http://127.0.0.1:30252/
```

В ответ возвращается стандартная страница:

```
Welcome to nginx!
```

и HTTP-код 200.

4. Мониторинг Prometheus

Что реализовано:

Prometheus развернут через Helm в отдельном namespace monitoring.

Дополнительно работают:

```
Prometheus Server  
Node Exporter  
kube-state-metrics
```

Как проверить:

```
kubectl get pods -n monitoring
```

Для доступа к Prometheus:

```
kubectl port-forward -n monitoring svc/prometheus-server 9090:80
```

Проверка готовности:

```
curl http://127.0.0.1:9090/-/ready
```

Для проверки сбора метрик используется запрос:

```
up
```

Значения `up = 1` показывают, что Prometheus успешно получает метрики от доступных targets.

5. Сбор логов Fluentd

Что реализовано:

Fluentd развернут в Kubernetes как DaemonSet.

Он собирает контейнерные логи Nginx из:

```
/var/log/containers/nginx-*_hackathon_*.log
```

В демонстрационной конфигурации собранные записи выводятся в stdout Fluentd.

Как проверить:

Сначала отправить запрос приложению:

```
curl http://127.0.0.1:30252/
```

После этого посмотреть Fluentd:

```
kubectl logs -n hackathon daemonset/fluentd --tail=50
```

В логах появляется запрос:

```
GET / HTTP/1.1
```

с HTTP-кодом:

```
200
```

Таким образом подтверждается, что Fluentd действительно получает access-логи Nginx.

Автоматизация

Для воспроизводимого развертывания подготовлены три Shell-скрипта.

bootstrap.sh — подготавливает Ubuntu и создает Kubernetes-кластер через kubeadm.

deploy.sh — автоматически устанавливает и разворачивает:

```
Calico  
Envoy Gateway  
Nginx  
Gateway API  
Fluentd  
Prometheus
```

check.sh — выполняет быструю проверку состояния Kubernetes, приложения, Gateway API, Prometheus, Fluentd и Envoy Gateway.

Главная особенность решения

Главной особенностью является полностью работающая цепочка прохождения HTTP-запроса:

```
Пользователь
↓
Envoy Gateway
↓
HTTPRoute
↓
Service
↓
Nginx
```

Работоспособность маршрутизации подтверждена реальным HTTP-запросом с ответом Nginx 200 OK.

Дополнительно реализованы мониторинг инфраструктуры через Prometheus и централизованный сбор контейнерных логов Nginx через Fluentd.

Самое сложное решение

Основная сложность возникла при публикации Envoy Gateway в локальном Kubernetes-кластере.

В облачной инфраструктуре Envoy может получить внешний IP через LoadBalancer. Однако локальный kubernetes-кластер в VirtualBox не предоставляет облачный LoadBalancer.

Поэтому для демонстрационного стенда доступ к Envoy организован через автоматически созданный NodePort. Это позволило проверить фактическое прохождение HTTP-трафика через Gateway API без подключения дополнительной облачной инфраструктуры.

Возможности дальнейшего развития

В дальнейшем решение можно расширить:

- добавить несколько control-plane и worker nodes для отказоустойчивости;
 - использовать внешний LoadBalancer или MetalLB;
 - добавить TLS и DNS для Gateway;
 - добавить persistent storage для Prometheus;
 - подключить Grafana для визуализации метрик;
 - подключить Alertmanager;
 - направлять Fluentd в Loki, Elasticsearch или OpenSearch;
 - добавить readiness/liveness probes;
 - настроить resource requests и limits;
 - добавить NetworkPolicy и RBAC;
 - реализовать CI/CD через GitHub Actions.
-

Итог

В проекте реализованы и проверены:

Kubernetes
kubeadm
containerd
Calico
Nginx
Kubernetes Service
Envoy Gateway
Gateway API
GatewayClass
Gateway
HTTPRoute
Prometheus
Node Exporter
kube-state-metrics
Fluentd
автоматизированное развертывание